SGCD - Guía Técnica de Endpoints Restantes

BRANCH: DEVELOP · BACKEND

Especificación de Integración: Hono.js Context (c.req) ↔ Supabase Stored Procedures (RPC)

> **Propósito de este documento:** Servir de especificación clara y estandarizada para que el equipo desarrolle los endpoints restantes del backend sin dudas de tipos o mapeos. Para cada endpoint se detalla exactamente qué datos llegan en `c.req` (parámetros de ruta, query params y request body en TypeScript) y qué función RPC de Supabase se debe invocar con su correspondiente firma y nombres en `snake_case` (`p_...`).

1. Controller (`controller.ts`)

Recibe `c: Context` , extrae datos de `c.req` , valida parámetros obligatorios y llama al servicio. Devuelve siempre `c.json({ success, data | error }, status)` .

2. Service (`service.ts`)

Conecta a Supabase con `createSupabaseUser(env, token)` . Invoca `supabase.rpc('nombre_rpc', { p_param: valor })` mapeando de TypeScript a SQL. Retorna `{ data, error }` .

3. Router (`routes.ts`)

Crea `new Hono()` con los tipos `Bindings` y `Variables` . Protege rutas con `authMiddleware` y define los verbos (`GET` , `POST` , `PATCH`).

ENDPOINTS RESTANTES A DESARROLLAR

GET /api/citas

Listar Citas del Usuario Autenticado (Historial y Próximas)

Qué llega en `c.req` (Petición)

Fuente / Variable	Tipo TS	Req?	Descripción
`c.req.header('Authorization')`	`string`	Sí	Token Bearer
`c.req.query('estado')`	`string`	No	Filtro estado
`c.req.query('limit')`	`string`	No	Default: `20`
`c.req.query('offset')`	`string`	No	Default: `0`

Supabase RPC a Invocar

`rpc_get_mis_citas`

Parámetro RPC	Tipo SQL	Mapeo TypeScript
`p_estado`	`TEXT`	`estado || null`
`p_limit`	`INTEGER`	`parseInt(limit || '20', 10)`
`p_offset`	`INTEGER`	`parseInt(offset || '0', 10)`

💡 **Nota de Implementación:** Usado en `CitasScreen.tsx` y `DashboardScreen.tsx` (ej. `/api/citas?limit=5`). La RPC ya resuelve la identidad mediante `auth.uid()` y hace los JOINs con paciente, doctor, procedimiento y sucursal.

POST /api/citas

Agendar una Nueva Cita Médica

Qué llega en `c.req` (Body JSON)

Campo en `c.req.json()`	Tipo TS	Req?	Ejemplo
`body.pacienteId`	`UUID`	Sí	`"b4f5..."`
`body.doctorId`	`UUID`	Sí	`"d725..."`
`body.procedimientoId`	`UUID`	Sí	`"1deb..."`
`body.sucursalId`	`UUID`	Sí	`"52a9..."`
`body.fechaHoraInicio`	`ISO string`	Sí	`"2026-10-25..."`
`body.notasCliente`	`string`	No	`"Molestia"`

Supabase RPC a Invocar

`rpc_agendar_cita`

Parámetro RPC	Tipo SQL	Mapeo TypeScript
`p_paciente_id`	`UUID`	`body.pacienteId`
`p_doctor_id`	`UUID`	`body.doctorId`
`p_procedimiento_id`	`UUID`	`body.procedimientoId`
`p_sucursal_id`	`UUID`	`body.sucursalId`
`p_fecha_hora_inicio`	`TIMESTAMPTZ`	`body.fechaHoraInicio`
`p_notas_cliente`	`TEXT`	`body.notasCliente || null`

💡 **Validaciones SQL automáticas:** La RPC valida horario laboral (Lun-Vie, 9:00 - 16:00 CDMX), que el doctor no tenga citas empalmadas y calcula la fecha de fin según el procedimiento. Retorna `{ cita_id, fecha_hora_fin }` .

SGCD - Guía Técnica de Endpoints Restantes

BRANCH: DEVELOP · BACKEND

Especificación de Integración: Hono.js Context (c.req) ↔ Supabase Stored Procedures (RPC)

PATCH /api/citas/:id/cancelar

Cancelar una Cita Activa

Qué llega en `c.req`

Fuente / Campo	Tipo TS	Req?	Descripción
`c.req.param('id')`	`UUID`	Sí	ID cita en URL
`body.motivo`	`string`	No	Motivo

Supabase RPC a Invocar

`rpc_cancelar_cita`

Parámetro RPC	Tipo SQL	Mapeo TypeScript
`p_cita_id`	`UUID`	`c.req.param('id')`
`p_motivo`	`TEXT`	`body?.motivo || null`

Consumido por: `CitaDetailScreen.tsx` al hacer clic en "Cancelar cita". La RPC verifica permisos (solo el creador o staff) y valida que no esté ya completada o cancelada. Retorna `{ success: true, cita_id }`.

POST /api/citas/:id/resumen

Cargar Resumen Médico / Completar Consulta (Doctor o Admin)

Qué llega en `c.req`

Fuente / Campo	Tipo TS	Req?	Descripción
`c.req.param('id')`	`UUID`	Sí	ID cita en URL
`body.resumen`	`string`	Sí	Diagnóstico / Receta

Supabase RPC a Invocar

`rpc_doctor_subir_resumen`

Parámetro RPC	Tipo SQL	Mapeo TypeScript
`p_cita_id`	`UUID`	`c.req.param('id')`
`p_resumen`	`TEXT`	`body.resumen`

Seguridad en DB: La RPC valida que el usuario autenticado sea el doctor de la cita (`d.perfil_id = auth.uid()`) o admin (`clinica.is_admin()`). Cambia automáticamente el estado a `'completada'`.

GET /api/citas/:id

Obtener Detalle Individual de una Cita

Qué llega en `c.req`

Fuente / Campo	Tipo TS	Req?	Desc
`c.req.param('id')`	`UUID`	Sí	ID cita

Estrategia en el Service

Como `rpc_get_mis_citas` ya devuelve objetos formateados con joins:

- Llamar `rpc_get_mis_citas` y buscar con `.find(c ⇒ c.id ═══ id)`.
- O consultar la tabla `citas` con `createSupabaseAdmin(env)` validando que pertenezca al usuario.

Consumido por: `CitaDetailScreen.tsx` para pintar toda la información, mapa de la sucursal y notas de la consulta.

POST /api/pacientes

Registrar Paciente o Familiar Dependiente

Qué llega en `c.req` (Body JSON)

Campo en `c.req.json()`	Tipo TS	Req?	Desc
`body.nombre`	`string`	Sí	Nombre
`body.apellidoPaterno`	`string`	Sí	A. Paterno
`body.apellidoMaterno`	`string`	No	A. Materno
`body.fechaNacimiento`	`YYYY-MM-DD`	No	Nacimiento
`body.genero`	`enum`	No	Género
`body.esFamiliar`	`boolean`	No	Def: `true`

Inserción en DB (Table: `clinica.pacientes`)

Columna SQL	Valor Asignado
`perfil_id`	`userId` del token (garantiza propiedad)
`nombre` / `apellido_paterno`	`body.nombre` / `body.apellidoPaterno`
`es_familiar`	`body.esFamiliar ?? true`
`familiar_de`	`body.familiarDe ?? null`

SGCD - Guía Técnica de Endpoints Restantes

BRANCH: DEVELOP · BACKEND

Cheat Sheet de Mapeo Rápido y Guía de Buenas Prácticas del Backend

CHEAT SHEET: MAPEO RÁPIDO DE NOMBRES TS ↔ POSTGRES RPC

Endpoint	Entrada TypeScript (c.req)	RPC de Supabase	Parámetros SQL (p_...)
`GET /api/citas`	`query: estado, limit, offset`	`rpc_get_mis_citas`	`p_estado, p_limit, p_offset`
`POST /api/citas`	`body: pacienteId, doctorId, procedimientoId, sucursalId, fechaHoraInicio, notasCliente`	`rpc_agendar_cita`	`p_paciente_id, p_doctor_id, p_procedimiento_id, p_sucursal_id, p_fecha_hora_inicio, p_notas_cliente`
`PATCH /api/citas/:id/cancelar`	`param: id, body: motivo`	`rpc_cancelar_cita`	`p_cita_id, p_motivo`
`POST /api/citas/:id/resumen`	`param: id, body: resumen`	`rpc_doctor_subir_resumen`	`p_cita_id, p_resumen`
`GET /api/citas/disponibilidad` ✓ Ya implementado	`query: doctorId, fecha, duracion`	`rpc_get_disponibilidad`	`p_doctor_id, p_fecha, p_duracion_minutos`
`GET /api/pacientes` ✓ Ya implementado	`header: Authorization`	`rpc_get_mis_pacientes`	*Sin params (usa `auth.uid()`)*

BUENAS PRÁCTICAS Y ERRORES CRÍTICOS A EVITAR

⚠️ **Errores comunes detectados que rompen la compilación o ejecución en Cloudflare Workers:**

- **No importar código de `frontend/` en el Worker:** El worker corre en Cloudflare V8 runtime. Si importas desde `../../../../frontend/lib/supabase/client`, el build colapsará con errores de dependencias de navegador / Next.js. Usa siempre `../../lib/supabase`.
- **Instanciar Hono con `new`:** Escribir `export const citasRoutes = Hono<...>()` lanza `TypeError: Class constructor Hono cannot be invoked without 'new'`. Siempre escribe `new Hono<...>()`.
- **Sintaxis de importación de tipos:** Recuerda usar `import type { Env } from ...` (no `import types`) y no agregar la extensión `.ts` en los paths de importación para mantener compatibilidad con TypeScript.
- **Ortografía en la respuesta JSON:** El frontend valida `if (json.success)`. Si envías `{ succes: true }` (con una sola 's'), el frontend lo evaluará como `undefined` y fallará.
- **Montar siempre el router en `index.ts`:** Crear los endpoints en `citas/routes.ts` o `pacientes/routes.ts` no basta: debes montar la ruta en `worker/src/index.ts` usando `app.route('/api/citas', citasRoutes)` para que Cloudflare Workers responda a las peticiones.

📌 **Estructura Estándar de Respuesta HTTP en SGCD:**

- Éxito: `c.json({ success: true, data: ... }, 200)` (o `201` para creación).
- Error de validación: `c.json({ success: false, error: 'Mensaje descriptivo' }, 400)`.
- Error de autorización: `c.json({ success: false, error: 'Token inválido o expirado' }, 401)`.